

# TEMA 6. Genericidad

---

1. Empleando **void\*** en C o **Object** en Java, pon un ejemplo de una estructura de datos, que empleando un array primitivo, permita alojar cualquier tipo de dato.

Te pongo **dos ejemplos clásicos**, uno en **C con void\*** y otro en **Java con Object**, porque precisamente estos enfoques *no genéricos* fueron el antecedente directo de la **genericidad**.

---



## Ejemplo en C usando void\*

---

En C, **void\*** representa un **puntero genérico** que puede apuntar a cualquier tipo de dato.

### Estructura de datos: array genérico

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    void** datos;    // array de punteros genéricos
    int capacidad;
    int tamaño;
} ArrayGenerico;
```

### Inicialización

```
ArrayGenerico* crearArray(int capacidad) {
    ArrayGenerico* a = malloc(sizeof(ArrayGenerico));
    a->capacidad = capacidad;
    a->tamaño = 0;
    a->datos = malloc(sizeof(void*) * capacidad);
    return a;
}
```

### Insertar elementos

```
void insertar(ArrayGenerico* a, void* elemento) {
    if (a->tamaño < a->capacidad) {
        a->datos[a->tamaño++] = elemento;
    }
}
```

```
}  
}
```

## Uso

```
int main() {  
    ArrayGenerico* a = crearArray(3);  
  
    int x = 10;  
    double y = 3.14;  
    char c = 'A';  
  
    insertar(a, &x);  
    insertar(a, &y);  
    insertar(a, &c);  
  
    printf("%d\n", *(int*)a->datos[0]);    // cast explícito  
    printf("%f\n", *(double*)a->datos[1]);  
    printf("%c\n", *(char*)a->datos[2]);  
  
    return 0;  
}
```

### ● Problemas de este enfoque (muy importantes)

- ✗ No hay **seguridad de tipos**
- ✗ Hace falta **casting manual**
- ✗ Errores detectados **en ejecución**, no en compilación

👉 Este diseño demuestra **por qué surgieron los genéricos en lenguajes modernos**.



## Ejemplo en **Java** usando **Object**

En Java, **todas las clases heredan de Object**, lo que permite almacenar cualquier tipo de objeto.

### Estructura de datos: array basado en **Object**

```
public class ArrayGenerico {  
  
    private Object[] datos;  
    private int tamaño;  
  
    public ArrayGenerico(int capacidad) {
```

```
        datos = new Object[capacidad];
        tamaño = 0;
    }

    public void insertar(Object o) {
        datos[tamaño++] = o;
    }

    public Object obtener(int i) {
        return datos[i];
    }
}
```

---

## Uso

```
public class Main {
    public static void main(String[] args) {

        ArrayGenerico a = new ArrayGenerico(3);

        a.insertar(10);           // Integer (autoboxing)
        a.insertar(3.14);        // Double
        a.insertar("Hola");      // String

        int x = (int) a.obtener(0);    // cast obligatorio
        double d = (double) a.obtener(1);
        String s = (String) a.obtener(2);

        System.out.println(x);
        System.out.println(d);
        System.out.println(s);
    }
}
```

---

### ● Problemas en Java sin genéricos

- ✗ Casting explícito obligatorio
- ✗ Posibles `ClassCastException`
- ✗ Errores en **tiempo de ejecución**
- ✗ Código menos legible



## Relación directa con la **genericidad**

Estos ejemplos muestran exactamente **el problema que resuelve la genericidad**:

| Enfoque                     | Problema principal                 |
|-----------------------------|------------------------------------|
| <code>void*</code> en C     | Total ausencia de control de tipos |
| <code>Object</code> en Java | Pérdida de seguridad de tipos      |

✓ Los **genéricos** permiten:

- Mantener **flexibilidad**
- Garantizar **seguridad de tipos en compilación**
- Eliminar castings

Ejemplo equivalente moderno en Java:

```
ArrayList<Integer> lista = new ArrayList<>();
```

## ✓ Resumen perfecto para examen

- En C, `void*` permite crear estructuras que almacenan cualquier tipo.
- En Java, `Object` cumple una función similar.
- Ambos enfoques permiten flexibilidad, pero **no son seguros**.
- La falta de control de tipos provocó la aparición de la **genericidad**.
- Los genéricos solucionan estos problemas en tiempo de compilación.

👉 Frase redonda:

*Antes de la genericidad, lenguajes como C y Java usaban `void*` u `Object`, sacrificando la seguridad de tipos a cambio de flexibilidad.*

## 2. Brevemente, ¿Qué significa la **programación genérica**? ¿Es el ejemplo anterior un ejemplo básico de programación genérica?

### ✓ ¿Qué significa la **programación genérica**?

La **programación genérica** es un paradigma que consiste en **definir algoritmos y estructuras de datos independientes del tipo concreto de los datos con los que trabajan**, de forma que:

- El **tipo se parametriza** (por ejemplo, `T`, `E`, `K`)
- El **compilador garantiza la seguridad de tipos**
- El mismo código sirve para **muchos tipos distintos**, sin duplicarlo

En Java, esto se materializa mediante **genéricos**:

```
class Caja<T> {  
    private T valor;
```

```
public void guardar(T v) { valor = v; }  
public T obtener() { return valor; }  
}
```

👉 Aquí las operaciones se definen **una sola vez**, y el tipo concreto se decide **en tiempo de compilación**, no en ejecución.

? ¿El ejemplo anterior con `void*` en C o `Object` en Java es programación genérica?

✗ **No, estrictamente NO es programación genérica**, aunque se le parece.

🔗 **Por qué no lo es:**

- El tipo **no está parametrizado**
- No hay **comprobación de tipos en compilación**
- Se necesitan **castings explícitos**
- Los errores aparecen **en tiempo de ejecución**

Es decir:

- `void*` en C
- `Object` en Java

permiten **flexibilidad**, pero **pierden seguridad de tipos**.

✅ Entonces, ¿qué es realmente ese ejemplo?

👉 Es un **antecedente** o una **solución precursora** a la programación genérica.

Se suele describir como:

- "programación genérica artesanal"
- "pseudo-genericidad"
- "programación basada en tipos raíz o punteros genéricos"

Pero **no cumple** la idea central de la programación genérica moderna.

vs Diferencia clave (muy típica de examen)

| Enfoque                                  | Seguridad de tipos | Castings | Errores detectados |
|------------------------------------------|--------------------|----------|--------------------|
| <code>void*</code> / <code>Object</code> | ✗ No               | ✅ Sí     | En ejecución       |
| Genéricos ( <code>&lt;T&gt;</code> )     | ✅ Sí               | ✗ No     | En compilación     |

💡 Resumen perfecto para examen

- La **programación genérica** permite escribir código reutilizable **parametrizado por tipos**.
- Garantiza **seguridad de tipos en compilación**.

- Los ejemplos con `void*` (C) o `Object` (Java) **no son programación genérica real**.
- Son soluciones previas que inspiraron la aparición de los **genéricos modernos**.
- Los genéricos eliminan castings y errores en tiempo de ejecución.

👉 Frase redonda para cerrar:

*La programación genérica no consiste solo en aceptar cualquier tipo, sino en hacerlo de forma segura y comprobable en tiempo de compilación.*

### 3. Indica los problemas respecto al chequeo de tipos, de emplear `void*` o `Object` cuando se crean estructuras de datos genéricas.

#### ✓ Problemas de chequeo de tipos usando `void*` (C) o `Object` (Java)

##### 1. ✗ Pérdida total de seguridad de tipos en compilación

El compilador **no puede comprobar** que los tipos que se insertan o recuperan de la estructura son correctos.

- En C, `void*` puede apuntar literalmente a **cualquier tipo**.
- En Java, `Object` acepta cualquier objeto.

👉 El compilador **no detecta errores de tipo**, aunque el programa sea incorrecto.

##### 2. 🔄 Necesidad de *casting* explícito

Al recuperar un elemento, el programador debe hacer un **casting manual** al tipo esperado.

```
Integer x = (Integer) array.obtener(0);
```

```
int valor = *(int*)array->datos[0];
```

✓ El compilador lo permite

✗ Pero no puede comprobar si el casting es correcto

##### 3. ✨ Errores detectados solo en tiempo de ejecución

Si el tipo real del objeto **no coincide** con el tipo al que se hace el casting:

- En **Java** → `ClassCastException`
- En **C** → comportamiento indefinido o error grave (segmentation fault, corrupción de memoria)

Ejemplo típico en Java:

```
a.insertar("Hola");  
Integer x = (Integer) a.obtener(0); // ClassCastException
```

👉 El error **no se detecta al compilar**, sino cuando el programa ya se está ejecutando.

---

#### 4. 🧑 El programador debe “recordar” los tipos

La estructura de datos **no guarda información fiable del tipo esperado** para cada posición.

Esto implica:

- Dependencia de documentación externa
- Mayor probabilidad de errores humanos
- Código frágil y difícil de mantener

En otras palabras:

“Confía en que el programador no se equivoque”.

---

#### 5. 🚫 Imposibilidad de expresar restricciones de tipo

No se puede indicar:

- “esta estructura solo admite **Integer**”
- “estos elementos deben implementar cierta interfaz”

Todo entra en el mismo saco (**void\*** o **Object**), sin control semántico.

---

#### 6. 📉 Código menos legible y menos expresivo

El uso constante de castings:

- Ensucia el código
- Dificulta su comprensión
- Oculta errores reales

Comparación clara:

```
// Sin genéricos  
Object o = lista.get(0);  
Integer i = (Integer) o;  
  
// Con genéricos  
Integer i = lista.get(0);
```

---

## 🗨️ Resumen comparativo (ideal para examen)

| Problema         | <code>void*</code> / <code>Object</code> |
|------------------|------------------------------------------|
| Chequeo de tipos | ❌ No en compilación                      |
| Castings         | ✅ Obligatorios                           |
| Errores de tipo  | ❌ En ejecución                           |
| Seguridad        | ❌ Baja                                   |
| Mantenibilidad   | ❌ Mala                                   |

### ✅ Conclusión clave

El uso de `void*` en C o `Object` en Java **permite flexibilidad**, pero **rompe el chequeo de tipos estático**, desplazando los errores de la compilación a la ejecución.

This is exactly **el problema que resuelve la programación genérica**:

- seguridad de tipos
- eliminación de castings
- detección temprana de errores

#### 👉 Frase redonda para cerrar:

*Las estructuras basadas en `void*` o `Object` sacrifican la seguridad de tipos, lo que motivó la introducción de los genéricos modernos.*

## 4. Vamos entonces con mecanismos de mejora de la programación genérica ¿Qué son los **parámetros de tipo**?

### ✅ ¿Qué son los **parámetros de tipo**?

En **programación genérica**, los **parámetros de tipo** son **marcadores de posición para tipos**, que se usan para **parametrizar clases, interfaces o métodos**.

Normalmente se escriben como letras mayúsculas (`T`, `E`, `K`, `V`, etc.) y **representan un tipo que se decidirá más adelante**, en tiempo de compilación.

#### 👉 Dicho de forma sencilla:

Un parámetro de tipo permite escribir código que funciona con distintos tipos **sin perder seguridad de tipos**.

### ✅ ¿Para qué sirven?

Los parámetros de tipo sirven para:

- ✅ **Escribir código reutilizable**, válido para muchos tipos

- ☒ Evitar el uso de **Object** y castings
- ☒ Detectar errores de tipo en compilación
- ☒ Hacer el código más expresivo y legible

---

## ☒ Ejemplo básico en Java

Clase genérica con un parámetro de tipo **T**

```
public class Caja<T> {  
  
    private T contenido;  
  
    public void guardar(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T obtener() {  
        return contenido;  
    }  
}
```

Aquí:

- **T** es el **parámetro de tipo**
- No se indica qué tipo concreto es todavía

---

## Uso de la clase genérica

```
Caja<Integer> cajaEnteros = new Caja<>();  
cajaEnteros.guardar(10);  
Integer x = cajaEnteros.obtener(); // sin casting
```

o

```
Caja<String> cajaTexto = new Caja<>();  
cajaTexto.guardar("Hola");  
String s = cajaTexto.obtener();
```

- ☒ El **mismo código** funciona con distintos tipos
- ☒ El compilador garantiza que no hay errores de tipo

---

## ☒ ¿Dónde pueden aparecer los parámetros de tipo?

## 1. En **clases**

```
class Lista<T> { ... }
```

## 2. En **interfaces**

```
interface Comparable<T> { ... }
```

## 3. En **métodos**

```
public static <T> T identidad(T valor) {  
    return valor;  
}
```

---

## ✓ Convenciones habituales

| Letra | Significado típico |
|-------|--------------------|
| T     | Type               |
| E     | Element            |
| K     | Key                |
| V     | Value              |
| N     | Number             |

(No son obligatorias, pero sí **buenas prácticas**)

---

## ✓ Relación con los problemas anteriores

Antes (con **Object** o **void\***):

- ✗ el tipo no estaba controlado
- ✗ castings obligatorios
- ✗ errores en ejecución

Con **parámetros de tipo**:

- ✓ el tipo está explícito
- ✓ no hay castings
- ✓ errores detectados en compilación

Ejemplo comparativo:

```
// Sin genéricos
Object o = lista.get(0);
Integer i = (Integer) o;

// Con parámetros de tipo
Integer i = lista.get(0);
```

## Resumen perfecto para examen

- Los **parámetros de tipo** permiten definir clases, interfaces o métodos **independientes del tipo concreto**.
- Se representan mediante letras como **T, E, K**.
- El tipo real se especifica **al usar la clase**, no al definirla.
- Permiten **reutilización de código, seguridad de tipos** y eliminan castings.
- Son la base de la **programación genérica moderna**.

👉 Frase redonda para cerrar:

*Los parámetros de tipo permiten escribir código genérico seguro, sustituyendo el uso de **Object** por tipos comprobados en compilación.*

5. En Java existe "generics", en C++ existen "templates". Pon un ejemplo de uso de programación genérica en ambos, instanciando una lista o vector dinámico que solo admite **String**. Introduce valores, y luego haz un recorrido de ellos mostrando cómo cada elemento es del tipo concreto con seguridad.

## Ejemplo en **Java** (Generics)

En Java usaremos **ArrayList<String>**, que es una **clase genérica** de la biblioteca estándar.

## Creación e inserción de elementos

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<String> lista = new ArrayList<>();

        lista.add("Hola");
        lista.add("Mundo");
```

```
lista.add("Java");

for (String s : lista) {
    System.out.println(s.toUpperCase());
}
}
```

---

## Características importantes

- ☒ `List<String>` solo admite **String**
- ☒ El compilador detecta errores:

```
lista.add(10); // ERROR de compilación
```

- ☒ No hay castings
- ☒ Cada elemento recuperado es **String garantizado**

👉 El tipo se comprueba **en compilación**, no en ejecución.

---

## ☒ Ejemplo en C++ (Templates)

En C++ usaremos `std::vector<std::string>`, que es un **template** de la STL.

---

## Creación e inserción de elementos

```
#include <iostream>
#include <vector>
#include <string>

int main() {

    std::vector<std::string> v;

    v.push_back("Hola");
    v.push_back("Mundo");
    v.push_back("C++");

    for (const std::string& s : v) {
        std::cout << s << std::endl;
    }

    return 0;
}
```

## Características importantes

- ✓ `std::vector<std::string>` solo admite `std::string`
- ✓ Intentar insertar otro tipo produce error de compilación:

```
v.push_back(10); // ERROR de compilación
```

- ✓ No hay conversiones inseguras
- ✓ Cada elemento es del tipo concreto `std::string`



## Comparación clave Java vs C++

| Aspecto              | Java (Generics)                      | C++ (Templates)                   |
|----------------------|--------------------------------------|-----------------------------------|
| Mecanismo            | Generics                             | Templates                         |
| Tipo definido        | En compilación                       | En compilación                    |
| Seguridad de tipos   | ✓ Sí                                 | ✓ Sí                              |
| Castings             | ✗ No                                 | ✗ No                              |
| Contenedor usado     | <code>ArrayList&lt;String&gt;</code> | <code>vector&lt;string&gt;</code> |
| Detección de errores | Compilación                          | Compilación                       |



## Idea fundamental sobre seguridad de tipos

---

En **ambos lenguajes**:

- La estructura está **parametrizada por tipo**
- El contenedor **rechaza tipos incorrectos**
- El código es:
  - más seguro
  - más legible
  - más mantenible

Esto contrasta directamente con enfoques antiguos basados en:

- `Object` (Java)
- `void*` (C)



## Resumen perfecto para examen

---

- Java utiliza **generics** (`ArrayList<String>`)
- C++ utiliza **templates** (`vector<string>`)
- Ambos permiten crear estructuras de datos genéricas **con seguridad de tipos**
- El tipo admitido se comprueba **en tiempo de compilación**
- No son necesarios *castings*
- Los errores de tipo no llegan a ejecutarse

👉 Frase redonda:

*Generics en Java y templates en C++ permiten escribir código genérico, reutilizable y seguro en tiempo de compilación.*

---

6. Sobre el funcionamiento de la programación genérica. ¿Qué hace el compilador cuando se instancia una clase que tiene parámetros de tipo? ¿Hace lo mismo C++ y Java? ¿Qué es el "type erasure" de Java y la "instanciación de plantillas" de C++?



¿Qué hace el compilador cuando se instancia una clase genérica?

Cuando usamos una clase genérica, como:

```
ArrayList<String> lista = new ArrayList<>();
```

o en C++:

```
std::vector<std::string> v;
```

el compilador **tiene que decidir cómo manejar ese parámetro de tipo** (`String`, `std::string`).

👉 **Java y C++ NO hacen lo mismo.**

## ◆ Java: **Type Erasure** (borrado de tipos)

### ✅ ¿Qué es el *type erasure*?

En **Java**, los **parámetros de tipo NO existen en tiempo de ejecución**.

Durante la compilación:

1. El compilador **comprueba los tipos** (seguridad de tipos).
2. Genera el *bytecode* **eliminando la información genérica**.
3. Sustituye los parámetros de tipo por:
  - `Object`
  - o por su **cota superior**, si existe (`<T extends Algo>`).

👉 Esto se llama **type erasure** (*borrado de tipos*).

### Ejemplo sencillo

Código fuente:

```
class Caja<T> {  
    private T valor;  
    public T get() { return valor; }  
}
```

Tras el *type erasure* (conceptualmente):

```
class Caja {  
    private Object valor;  
    public Object get() { return valor; }  
}
```

Y el compilador **inserta castings automáticamente** donde haga falta.

### Consecuencias del *type erasure*

✅ Ventajas:

- Compatibilidad con código antiguo (Java pre-genéricos)
- No incremento del tamaño del bytecode
- Runtime más simple

#### ✗ Limitaciones importantes:

- No se puede preguntar por `T` en tiempo de ejecución:

```
if (obj instanceof T) // ILEGAL
```

- No se pueden crear arrays genéricos:

```
new T[10]; // ILEGAL
```

- En runtime, `List<String>` y `List<Integer>` son exactamente iguales:

```
lista.getClass() // ambas son ArrayList
```

#### ✓ En resumen:

En Java, los genéricos son **solo una ayuda del compilador**, no del runtime.

---

## ◆ C++: Instanciación de plantillas (Template instantiation)

---

### ✓ ¿Qué hace C++ con los templates?

En C++, los **templates sí existen realmente**.

Cuando se instancia un template, el compilador:

1. **Genera código nuevo y específico** para cada tipo usado.
2. Cada combinación de tipos produce una **clase o función distinta**.
3. Todo existe **en tiempo de compilación**.

👉 A esto se le llama **instanciación de plantillas**.

---

### Ejemplo

Código:

```
template <typename T>
class Caja {
    T valor;
};
```

Uso:

```
Caja<int> c1;
Caja<std::string> c2;
```

El compilador genera (conceptualmente):

```
class Caja_int { int valor; };
class Caja_string { std::string valor; };
```

✓ Son **tipos distintos**, con memoria y código propios.

---

## Consecuencias en C++

✓ Ventajas:

- Seguridad de tipos total
- Mejor rendimiento (no castings)
- Información de tipo disponible en compilación
- Metaprogramación avanzada

✗ Inconvenientes:

- Código más grande (code bloat)
- Errores de compilación largos y complejos
- Recompilación necesaria al cambiar templates



## Comparación directa Java vs C++

| Aspecto               | Java (Generics) | C++ (Templates) |
|-----------------------|-----------------|-----------------|
| Momento del control   | Compilación     | Compilación     |
| Existencia en runtime | ✗ No            | ✓ Sí            |
| Técnica usada         | Type erasure    | Instanciación   |
| Código generado       | Uno solo        | Uno por tipo    |

| Aspecto                          | Java (Generics) | C++ (Templates) |
|----------------------------------|-----------------|-----------------|
| Información de tipo en ejecución | ✗ No            | ✓ Parcial       |
| Compatibilidad hacia atrás       | ✓ Sí            | ✗ No            |
| Rendimiento                      | Bueno           | Muy alto        |

✓ Entonces, ¿hace lo mismo el compilador en Java y en C++?

✗ No.

- Java:

Comprueba los tipos y luego **borra la información genérica**.

- C++:

Genera **implementaciones reales distintas** para cada tipo.

## 🗨️ Resumen perfecto para examen

- Al instanciar una clase genérica, el compilador garantiza la **seguridad de tipos**.
- **Java usa *type erasure***: los tipos genéricos desaparecen tras compilar.
- **C++ usa *instanciación de plantillas***: genera código específico por tipo.
- En Java, los genéricos son una **característica del compilador**.
- En C++, los templates son una **característica del lenguaje a bajo nivel**.

👉 Frase redonda para cerrar:

*Java comprueba los genéricos y los borra; C++ los materializa generando código nuevo.*

7. Vamos a crear una nueva clase con parámetros de tipo. Define en Java una clase **Par**, que permite alojar dos valores de tipos diferentes. Incluye un constructor y un getter para cada tipo. Pon un ejemplo de uso de ese **Par**, por ejemplo para especificar el tipo de retorno de una función que devuelve en un **Par** la media y desviación típica de un array de **double**.

✓ Clase genérica **Par**

La clase **Par** permitirá almacenar **dos valores de tipos distintos**, usando **dos parámetros de tipo**.

Definición de la clase

```
public class Par<T, U> {
    private T primero;
    private U segundo;
```

```
public Par(T primero, U segundo) {
    this.primerο = primero;
    this.segundo = segundo;
}

public T getPrimero() {
    return primero;
}

public U getSegundo() {
    return segundo;
}
}
```

## Claves importantes

- **T** y **U** son **parámetros de tipo**
- Permiten almacenar **valores heterogéneos**
- No se usa **Object**
- No hay *casting*
- El compilador garantiza la **seguridad de tipos**

---

## ✓ Ejemplo de uso: media y desviación típica

Vamos a crear una función que:

- Recibe un **double[]**
- Calcula:
  - la **media**
  - la **desviación típica**
- Devuelve ambos valores empaquetados en un **Par<Double, Double>**

---

## Método que realiza el cálculo

```
public class Estadisticas {

    public static Par<Double, Double> mediaYDesviacion(double[] valores) {

        double suma = 0.0;
        for (double v : valores) {
            suma += v;
        }
        double media = suma / valores.length;

        double sumaCuadrados = 0.0;
        for (double v : valores) {
            sumaCuadrados += Math.pow(v - media, 2);
        }
    }
}
```

```
    }  
    double desviacion = Math.sqrt(sumaCuadrados / valores.length);  
  
    return new Par<>(media, desviacion);  
  }  
}
```

## Uso del método

```
public class Main {  
    public static void main(String[] args) {  
  
        double[] datos = {2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0};  
  
        Par<Double, Double> resultado = Estadisticas.mediaYDesviacion(datos);  
  
        System.out.println("Media: " + resultado.getPrimero());  
        System.out.println("Desviación típica: " + resultado.getSegundo());  
    }  
}
```

## ✓ Qué demuestra este ejemplo

- ✓ Uso de **clase genérica con dos parámetros de tipo**
- ✓ Tipos distintos en un mismo objeto (**Double, Double**)
- ✓ Retorno múltiple limpio y seguro
- ✓ Sin arrays auxiliares
- ✓ Sin **Object**
- ✓ Sin castings
- ✓ Errores detectados en **compilación**

## 💬 Comparación conceptual (por qué es buena idea)

Antes (sin genéricos):

```
Object[] res = calcular(...);  
double media = (Double) res[0];  
double desv = (Double) res[1];
```

Ahora (con genéricos):

```
Par<Double, Double> res = calcular(...);  
double media = res.getPrimero();
```

```
double desv = res.getSegundo();
```

👉 **Más seguro, más legible y más expresivo**

## ✓ Resumen perfecto para examen

- Una clase genérica puede tener **varios parámetros de tipo**.
- `Par<T, U>` permite almacenar dos valores de tipos distintos.
- Los parámetros de tipo se sustituyen por tipos concretos **en compilación**.
- El compilador garantiza la **seguridad de tipos**.
- Los genéricos permiten devolver **múltiples valores relacionados** de forma limpia.
- Evitan el uso de `Object` y *casting* manual.

👉 **Frase redonda para cerrar:**

*Los parámetros de tipo permiten definir estructuras flexibles y seguras, como `Par<T, U>`, que encapsulan varios resultados manteniendo el chequeo de tipos en compilación.*

8. En Java, se pueden declarar parámetros de tipo también a nivel de método, no solo a nivel de clase. Pon un ejemplo con un método genérico `seleccionaUno`, que pasados dos objetos del mismo tipo, te devuelva aleatoriamente uno de ellos. Muestra la diferencia de definirlo con dos `Object`, a definirlo con dos parámetros de tipo, en terminos de (i) evitar downcasting y (ii) forzar que ambos objetos sean del mismo tipo.

## ✓ Método **NO** genérico usando `Object`

Primero, la versión "antigua", **sin programación genérica real**.

```
import java.util.Random;

public class Util {

    public static Object seleccionaUno(Object a, Object b) {
        Random r = new Random();
        return r.nextBoolean() ? a : b;
    }
}
```

Uso

```
Object o = Util.seleccionaUno("Hola", "Adiós");  
String s = (String) o; // downcasting obligatorio
```

## ✖ Problemas

### 1. Downcasting obligatorio

```
String s = (String) o;
```

→ posible `ClassCastException`

### 2. No fuerza que los objetos sean del mismo tipo

```
Object o = Util.seleccionaUno("Hola", 10); // COMPILA  
String s = (String) o; // ERROR en ejecución
```

👉 El compilador **no puede protegerse**.

## ✅ Método **genérico** con parámetro de tipo

Ahora la versión **correcta**, usando un **parámetro de tipo a nivel de método**.

```
import java.util.Random;  
  
public class Util {  
  
    public static <T> T seleccionaUno(T a, T b) {  
        Random r = new Random();  
        return r.nextBoolean() ? a : b;  
    }  
}
```

📌 Observa:

- `<T>` se declara **antes del tipo de retorno**
- El método es genérico aunque la clase no lo sea

## Uso

```
String s = Util.seleccionaUno("Hola", "Adiós");  
// NO hay casting
```

o:

```
Integer x = Util.seleccionaUno(3, 7);
```

## ✓ Comparación pedida explícitamente

### (i) Evitar *downcasting*

Con `Object` ✗

```
Object o = seleccionaUno("Hola", "Adiós");  
String s = (String) o; // casting manual
```

Con genéricos ✓

```
String s = seleccionaUno("Hola", "Adiós"); // sin casting
```

✓ El tipo devuelto es **exactamente T**

✓ Seguridad de tipos en **compilación**

### (ii) Forzar que ambos objetos sean del mismo tipo

Con `Object` ✗

Esto **compila** pero es incorrecto:

```
Object o = seleccionaUno("Hola", 10);
```

🔧 Error tarde o temprano en ejecución.

Con parámetro de tipo ✓

Esto **NO compila**:

```
String s = Util.seleccionaUno("Hola", 10);  
// ERROR de compilación
```

✅ El compilador exige que:

T sea el mismo tipo para ambos argumentos

👉 El error se detecta **antes de ejecutar el programa**.



## Clave conceptual importante

Este método es especialmente interesante porque:

- ✅ **La clase no es genérica**
- ✅ **El método sí lo es**
- ✅ El parámetro de tipo vive **solo en el método**
- ✅ Es más flexible y expresivo

Ejemplo típico de API real:

```
Collections.max(Collection<T>)  
Optional<T>  
Stream<T>.map(...)
```



## Resumen perfecto para examen

- En Java se pueden definir **parámetros de tipo a nivel de método**.
- Un método genérico se declara con **<T>** antes del tipo de retorno.
- Permite:
  1. ✅ Evitar *downcasting*
  2. ✅ Forzar coherencia de tipos entre parámetros
- Usar **Object**:
  - pierde seguridad de tipos
  - provoca errores en ejecución
- Usar genéricos:
  - detecta errores en compilación
  - produce código más claro y seguro

👉 **Frase redonda para cerrar:**

Los métodos genéricos permiten expresar relaciones de tipo entre parámetros y resultados, algo imposible usando *Object*.

9. ¿Se pueden establecer restricciones en los parámetros de tipo? Por ejemplo, si quiero definir un tipo genérico `<T>`, ¿puedo decir que tenga que ser, al menos, un número para poder tratarlo como tal? Pon un ejemplo en Java de un `Punto` con dos coordenadas, métodos `getX`, `getY`, y una función `calcularDistanciaA` otro `Punto`. Permite que esas coordenadas sean cualquier tipo de número. Pon dos soluciones: una simplemente creando coordenadas de tipo `Number` y otra añadiendo generics para reforzar el chequeo de tipos y saber exactamente con qué tipo de número trabaja el `Punto`. En este caso y respecto al "type erasure", ¿cuál es el tipo final tras la compilación?

### ✓ Restricciones en parámetros de tipo (bounded type parameters)

Sintaxis general:

```
<T extends TipoBase>
```

- `T` debe ser `TipoBase` o una subclase suya.
- El compilador permite usar **los métodos de** `TipoBase` sobre `T`.

Ejemplo típico:

```
<T extends Number>
```

## ◆ Solución 1: Usar directamente `Number` (sin genéricos)

Esta es la solución **simple**, pero **menos precisa en chequeo de tipos**.

Clase `Punto` con `Number`

```
public class Punto {  
  
    private Number x;  
    private Number y;  
  
    public Punto(Number x, Number y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

```

    public Number getX() {
        return x;
    }

    public Number getY() {
        return y;
    }

    public double calcularDistanciaA(Punto otro) {
        double dx = this.x.doubleValue() - otro.x.doubleValue();
        double dy = this.y.doubleValue() - otro.y.doubleValue();
        return Math.sqrt(dx * dx + dy * dy);
    }
}

```

## Uso

```

Punto p1 = new Punto(1, 2);           // Integer
Punto p2 = new Punto(3.5, 4.5);       // Double

double d = p1.calcularDistanciaA(p2);

```

## ⚠ Limitaciones de esta solución

- ☒ Funciona para **cualquier número**
- ☒ **Mezcla tipos numéricos sin control**
- ☒ No sabes si trabajas con `Integer`, `Double`, etc.
- ☒ El compilador **no impide incoherencias semánticas**

Ejemplo permitido (pero dudoso conceptualmente):

```

Punto p = new Punto(new Integer(1), new BigDecimal("2.3"));

```

## ◆ Solución 2: Usar genéricos con restricción (`<T extends Number>`)

Ahora usamos **programación genérica real**, reforzando el chequeo de tipos.

Clase genérica `Punto<T extends Number>`

```

public class Punto<T extends Number> {

```

```

private T x;
private T y;

public Punto(T x, T y) {
    this.x = x;
    this.y = y;
}

public T getX() {
    return x;
}

public T getY() {
    return y;
}

public double calcularDistanciaA(Punto<T> otro) {
    double dx = this.x.doubleValue() - otro.x.doubleValue();
    double dy = this.y.doubleValue() - otro.y.doubleValue();
    return Math.sqrt(dx * dx + dy * dy);
}
}

```

## Uso (mucho más seguro)

```

Punto<Integer> p1 = new Punto<>(1, 2);
Punto<Integer> p2 = new Punto<>(4, 6);

double d = p1.calcularDistanciaA(p2);

```

✓ Correcto.

✗ Error detectado en compilación

```

Punto<Integer> p1 = new Punto<>(1, 2);
Punto<Double> p2 = new Punto<>(3.5, 4.5);

p1.calcularDistanciaA(p2); // ERROR de compilación

```

👉 El compilador impide mezclar tipos de coordenadas.

## 🗨 Comparación clara de ambas soluciones

| Aspecto      | Number directo | Punto<T extends Number> |
|--------------|----------------|-------------------------|
| Flexibilidad | ✓ Alta         | ✓ Alta                  |

| Aspecto                | Number directo | Punto<T extends Number> |
|------------------------|----------------|-------------------------|
| Seguridad de tipos     | ✗ Menor        | ✓ Alta                  |
| Mezcla de tipos        | ✓ Permitida    | ✗ Prohibida             |
| Tipo conocido          | ✗ No           | ✓ Sí (T)                |
| Chequeo en compilación | ✗ Limitado     | ✓ Fuerte                |



## ¿Qué ocurre con el **type erasure** en este caso?

En Java, **los genéricos se borran tras la compilación**.

Código genérico original

```
public class Punto<T extends Number> {
    private T x;
    ...
}
```

Tras el *type erasure* (conceptualmente)

```
public class Punto {
    private Number x;
    private Number y;

    public Number getX() { ... }
    public Number getY() { ... }
}
```

### ✦ Regla clave del **type erasure**:

- T se sustituye por su **cota superior**
- Aquí la cota es **Number**
- Si no hubiera cota, sería **Object**

✓ El **chequeo fuerte ocurre en compilación**

✓ En ejecución, **todo es Number**



## Resumen perfecto para examen

- Sí, **se pueden restringir parámetros de tipo** usando **<T extends ...>**.
- **<T extends Number>** garantiza que **T** es numérico.
- Usar **Number** directamente es válido pero **menos seguro**.
- Los genéricos permiten:

- reforzar el chequeo de tipos
- evitar mezclas incoherentes
- conocer el tipo exacto (**Integer**, **Double**, etc.)
- Tras el **type erasure**, el tipo final es la **cota superior** (**Number**).

👉 **Frase redonda para cerrar:**

*Las restricciones en parámetros de tipo permiten combinar flexibilidad con seguridad, delegando el control de tipos al compilador aunque en ejecución todo se reduzca a la cota superior.*

10. Sobre las soluciones anteriores. Si bien ambas permiten trabajar con distintos tipos de número sin duplicar la clase **Punto**, reflexiona sobre el refuerzo del chequeo de tipos con generics. ¿Permiten ambas crear un punto con una coordenada de tipo entero y la otra coordenada de tipo real? ¿Qué tipo devuelve el **getX** con la solución sin generics y qué tipo devuelve el que tiene la solución con generics?

✅ Recordatorio rápido de las dos soluciones

- ◆ Solución A (sin genéricos): usar **Number**

```
class Punto {  
    private Number x;  
    private Number y;  
  
    public Punto(Number x, Number y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public Number getX() { return x; }  
    public Number getY() { return y; }  
}
```

- ◆ Solución B (con genéricos): **<T extends Number>**

```
class Punto<T extends Number> {  
    private T x;  
    private T y;  
  
    public Punto(T x, T y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public T getX() { return x; }  
}
```

```
public T getY() { return y; }  
}
```

✓ ¿Permiten ambas crear un punto con una coordenada entera y otra real?

♦ Solución **sin genéricos** (`Number`)

✓ Sí, lo permite.

```
Punto p = new Punto(3, 4.5); // Integer y Double
```

Esto es legal porque:

- `Integer` y `Double` son ambos subclases de `Number`
- El compilador **no impone ninguna relación** entre `x` e `y`

👉 Desde el punto de vista del compilador, **todo es `Number`**, así que no hay ningún problema.

✳ **Conclusión:**

Esta solución es **más permisiva**, pero **menos estricta** en chequeo de tipos.

♦ Solución **con genéricos** (`Punto<T extends Number>`)

✗ **No lo permite** (tal como está definida).

```
Punto<Integer> p = new Punto<>(3, 4.5); // ERROR de compilación
```

¿Por qué?

- `T` debe ser **un único tipo**
- Si `T` es `Integer`, **ambas coordenadas deben ser `Integer`**
- Si `T` es `Double`, **ambas deben ser `Double`**

✓ Correcto:

```
Punto<Integer> p1 = new Punto<>(3, 4);  
Punto<Double> p2 = new Punto<>(3.0, 4.5);
```

👉 El genérico fuerza coherencia de tipos dentro del objeto.

✳ **Conclusión:**

La versión genérica **refuerza el chequeo de tipos** e impide mezclar tipos numéricos accidentalmente.

## ✓ ¿Qué tipo devuelve `getX()` en cada solución?

Esta es la diferencia **clave** a nivel de API.

---

### ♦ `getX()` en la solución **sin genéricos**

```
public Number getX()
```

Por tanto:

```
Number x = p.getX();
```

Si quieres tratarlo como `Integer` o `Double`, necesitas *casting*:

```
Integer xi = (Integer) p.getX(); // posible ClassCastException
```

#### ✦ Características:

- El tipo concreto **se pierde**
  - El compilador **no sabe** si es `Integer`, `Double`, etc.
  - El *casting* es responsabilidad del programador
- 

### ♦ `getX()` en la solución **con genéricos**

```
public T getX()
```

Y si instancias:

```
Punto<Integer> p = new Punto<>(3, 4);
```

Entonces:

```
Integer x = p.getX(); // sin casting
```

#### ✦ Características:

- El tipo concreto **se conserva**
- El compilador **sabe exactamente qué tipo devuelve**

- No hay *downcasting*
- Mayor seguridad y claridad

## 🗨 Comparación directa (muy clara para examen)

| Aspecto                                | Sin genéricos ( <b>Number</b> ) | Con genéricos ( <b>&lt;T extends Number&gt;</b> ) |
|----------------------------------------|---------------------------------|---------------------------------------------------|
| Mezclar <b>Integer</b> y <b>Double</b> | ✅ Permitido                     | ❌ Prohibido                                       |
| Coherencia interna                     | ❌ No forzada                    | ✅ Forzada                                         |
| Tipo de <b>getX()</b>                  | <b>Number</b>                   | <b>T</b> (tipo concreto)                          |
| Necesidad de casting                   | ✅ Sí                            | ❌ No                                              |
| Seguridad de tipos                     | ❌ Menor                         | ✅ Mayor                                           |
| Expresividad de la API                 | ❌ Baja                          | ✅ Alta                                            |

## ✅ Conclusión clara

- **Ambas soluciones evitan duplicar la clase **Punto**** y permiten trabajar con números.
- La solución basada en **Number** es:
  - más flexible
  - pero pierde información de tipo
  - y delega errores al programador
- La solución con **genéricos**:
  - refuerza el chequeo de tipos
  - impide mezclar coordenadas de tipos distintos
  - conserva el tipo exacto en los getters
  - detecta errores **en compilación**

### 👉 Idea clave:

*Los genéricos no solo sirven para aceptar muchos tipos, sino para expresar relaciones y restricciones entre ellos con seguridad en compilación.*

11. Hagamos un ejemplo avanzado. El siguiente código, con interfaz **Punto**, que define un método **calcularDistanciaA(Punto p)**, junto con las implementaciones **Punto2D** y **Punto3D**. Añade generics para asegurarnos que la sobreescritura del método calcular distancia a otro **Punto** siempre es sobre un **Punto** del mismo tipo, evitando **instanceof** y el **downcasting**.

```
public interface Punto {
    public double distanciaA(Punto p);
}
```

```

public class Punto2D implements Punto {
    private final double x, y;
    public Punto2D(double x, double y) {
        this.x = x; this.y = y;
    }

    @Override
    public double distanciaA(Punto p) {
        if (p instanceof Punto2D) {
            Punto2D p2d = (Punto2D) p;
            return Math.sqrt(Math.pow(x - p2d.x, 2)
                + Math.pow(y - p2d.y, 2));
        } else {
            throw new RuntimeException("p debe ser Punto 2D");
        }
    }
}

public class Punto3D implements Punto {
    // Igual que Punto2D, pero con tres coordenadas
    ...
}

```

Este es un **ejemplo excelente y avanzado**, y la solución correcta pasa por usar lo que se conoce como **genéricos con autorreferencia**, también llamado **F-bounded polymorphism**.

Con este enfoque conseguimos exactamente lo que pides:

- ✓ Que **solo se pueda calcular la distancia entre puntos del mismo tipo**
- ✓ Eliminar **instanceof**
- ✓ Eliminar **downcasting**
- ✓ Mantener **polimorfismo y sobreescritura correcta**

## ✓ Idea clave del rediseño

En lugar de:

```
double distanciaA(Punto p);
```

usamos una **interfaz genérica donde el tipo se refiere a sí mismo**:

```
interface Punto<T extends Punto<T>>
```

Esto permite expresar en el **sistema de tipos** algo que antes no se podía expresar:

“Un **Punto2D** solo puede calcular distancia a otro **Punto2D**  
y un **Punto3D** solo a otro **Punto3D**”.

## ✓ Nueva interfaz **Punto** con genéricos

```
public interface Punto<T extends Punto<T>> {  
    double distanciaA(T p);  
}
```

Qué significa esto exactamente

- **T** es un subtipo de **Punto<T>**
- El método **distanciaA** **solo acepta el mismo tipo concreto**
- El compilador garantiza la coherencia
- No hace falta comprobar nada en runtime

## ✓ Implementación: **Punto2D**

```
public class Punto2D implements Punto<Punto2D> {  
  
    private final double x;  
    private final double y;  
  
    public Punto2D(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    @Override  
    public double distanciaA(Punto2D p) {  
        double dx = this.x - p.x;  
        double dy = this.y - p.y;  
        return Math.sqrt(dx * dx + dy * dy);  
    }  
}
```

Observa lo importante

- ✓ El método es ahora:

```
double distanciaA(Punto2D p)
```

- ✓ **No hay instanceof**
- ✓ **No hay casting**
- ✓ Sobreescritura 100% segura y clara

## ✓ Implementación: Punto3D

```
public class Punto3D implements Punto<Punto3D> {

    private final double x;
    private final double y;
    private final double z;

    public Punto3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public double distanciaA(Punto3D p) {
        double dx = this.x - p.x;
        double dy = this.y - p.y;
        double dz = this.z - p.z;
        return Math.sqrt(dx * dx + dy * dy + dz * dz);
    }
}
```

## ✓ Uso correcto (polimorfismo seguro)

```
Punto2D a = new Punto2D(0, 0);
Punto2D b = new Punto2D(3, 4);

double d2 = a.distanciaA(b); // ✓ OK
```

```
Punto3D c = new Punto3D(0, 0, 0);
Punto3D d = new Punto3D(1, 2, 2);

double d3 = c.distanciaA(d); // ✓ OK
```

## ✗ Uso incorrecto (error en compilación)

```
Punto2D p2 = new Punto2D(0, 0);
Punto3D p3 = new Punto3D(1, 1, 1);

p2.distanciaA(p3); // ✗ ERROR de compilación
```

👉 Este error **ya no llega a ejecutarse**, se detecta **en compilación**.

## 💬 Qué hemos conseguido exactamente

| Aspecto                 | Código original | Código con genéricos |
|-------------------------|-----------------|----------------------|
| Coherencia de tipos     | ✗ En runtime    | ✓ En compilación     |
| <code>instanceof</code> | ✓ Necesario     | ✗ Eliminado          |
| Downcasting             | ✓ Necesario     | ✗ Eliminado          |
| Seguridad               | ✗ Parcial       | ✓ Total              |
| Diseño                  | ✗ Defensivo     | ✓ Declarativo        |
| Polimorfismo            | ✓ Sí            | ✓ Sí (mejor)         |

## ✓ Relación con *type erasure*

A pesar de usar genéricos:

- **En runtime** Java los borra (*type erasure*)
- Pero **el chequeo fuerte ya se hizo en compilación**
- El runtime nunca recibe combinaciones incorrectas

Conceptualmente, tras el *type erasure*, el método queda como:

```
double distanciaA(Punto p);
```

👉 **La seguridad no desaparece**, porque el compilador ya impidió los usos ilegales.

## ✓ Nombre técnico de la técnica (muy de examen)

Este patrón se llama:

### F-bounded polymorphism

(polimorfismo acotado por autorreferencia)

Se usa en:

- `Enum<E> extends Enum<E>>`
- APIs matemáticas y geométricas
- Librerías extremadamente robustas

## ✓ Resumen perfecto para examen

- Se usa una **interfaz genérica autorreferenciada**  
`Punto<T extends Punto<T>>`
- Cada clase concreta se liga a **su propio tipo**
- Se elimina:
  - `instanceof`
  - downcasting
- El compilador garantiza que:
  - `Punto2D` solo opera con `Punto2D`
  - `Punto3D` solo con `Punto3D`
- Es un uso avanzado de genéricos para **reforzar el polimorfismo**

👉 **Frase redonda final:**

*Los genéricos permiten trasladar al compilador restricciones que antes solo podían comprobarse en tiempo de ejecución.*

12. Dado que `String` es subtipo de `Object`, ¿significa eso que `List<String>` es subtipo de `List<Object>`? ¿Y que `String[]` es subtipo de `Object[]`? Razona por qué la respuesta es diferente en cada caso y qué problema en tiempo de ejecución puede aparecer con los arrays. A partir de estos ejemplos, define qué significa que un tipo genérico sea **covariante**, **contravariante** o **invariante** respecto a su parámetro de tipo.

Esta es **una de las preguntas más importantes y finas de genéricos en Java**. Vamos por partes y razonando con calma, porque aquí se juntan **subtipado**, **genéricos**, **arrays** y **seguridad en tiempo de ejecución**.

✅ ¿Si `String`  $\subset$  `Object`, entonces `List<String>`  $\subset$  `List<Object>`?

❌ **NO**. En Java, `List<String>` **NO** es subtipo de `List<Object>`.

Aunque:

```
String extends Object // ✅ cierto
```

NO se cumple que:

```
List<String> extends List<Object> // ❌ falso
```

Ejemplo que **NO** compila

```
List<String> ls = new ArrayList<>();
```

```
List<Object> lo = ls; // ERROR de compilación
```

❓ ¿Por qué NO está permitido?

Porque sería **peligroso para la seguridad de tipos**.

Si se permitiera:

```
List<String> ls = new ArrayList<>();
List<Object> lo = ls;           // si esto estuviera permitido
lo.add(10);                    // añadir un Integer
```

Ahora `ls` contendría un `Integer`, aunque debería contener solo `String`.

```
String s = ls.get(0); // ✨ ClassCastException
```

👉 Para evitar este problema, Java define que:

**Los tipos genéricos en Java son invariantes** respecto a sus parámetros de tipo.

✅ ¿Y qué pasa con los arrays?

¿`String[]` es subtipo de `Object[]`?

✅ **SÍ. En Java, los arrays SON covariantes.**

```
String[] sa = new String[10];
Object[] oa = sa; // ✅ permitido
```

❗ Pero aquí aparece un problema en tiempo de ejecución

```
Object[] oa = new String[2];
oa[0] = "Hola"; // ✅ OK
oa[1] = 10;     // ❌ ArrayStoreException en tiempo de ejecución
```

👉 El compilador **lo permite**, pero la **JVM lo detecta en ejecución**.

Esto ocurre porque:

- El array **sabe en runtime** que es realmente `String[]`
- Al intentar guardar un `Integer`, la JVM lanza `ArrayStoreException`

## ✦ Conclusión clave:

Los arrays son covariantes, pero necesitan comprobaciones en tiempo de ejecución, lo que los hace menos seguros que los genéricos.

## 💡 ¿Por qué Java hizo esto?

- ☒ Los arrays existían **antes de los genéricos**
- ☒ Se priorizó compatibilidad hacia atrás
- ☒ A costa de mover errores a tiempo de ejecución

Con genéricos, Java decidió **no repetir ese error**.

## ☒ Comparación clara

| Caso                                                                      | Subtipado permitido                    | Error posible                                              |
|---------------------------------------------------------------------------|----------------------------------------|------------------------------------------------------------|
| <code>String</code> $\subset$ <code>Object</code>                         | <input checked="" type="checkbox"/> Sí | <input checked="" type="checkbox"/> No                     |
| <code>String[]</code> $\subset$ <code>Object[]</code>                     | <input checked="" type="checkbox"/> Sí | <input checked="" type="checkbox"/> En ejecución           |
| <code>List&lt;String&gt;</code> $\subset$ <code>List&lt;Object&gt;</code> | <input checked="" type="checkbox"/> No | <input checked="" type="checkbox"/> Evitado en compilación |

## ☒ Definiciones clave: covarianza, contravarianza e invariancia

Ahora podemos definir formalmente los conceptos.

### ◆ Covarianza

Un tipo genérico `G<T>` es **covariante** si:

Si  $A \subset B$ , entonces  $G<A> \subset G<B>$

### ✦ Ejemplo en Java:

- Arrays: ☒ covariantes

`String[]`  $\subset$  `Object[]`

### ☒ Problema:

- Puede causar errores **en tiempo de ejecución**

### ◆ Invariancia

Un tipo genérico  $G\langle T \rangle$  es **invariante** si:

Aunque  $A \subset B$ ,  
 $G\langle A \rangle$  NO es subtipo de  $G\langle B \rangle$

✦ Ejemplo en Java:

- $List\langle T \rangle$
- $Set\langle T \rangle$
- $Map\langle K, V \rangle$

$List\langle String \rangle \not\subset List\langle Object \rangle$

✓ Ventaja:

- **Seguridad total en compilación**
- Cero errores de tipo en runtime

👉 Java eligió invariancia para los genéricos.

---

### ◆ Contravarianza

Un tipo genérico  $G\langle T \rangle$  es **contravariante** si:

Si  $A \subset B$ , entonces  $G\langle B \rangle \subset G\langle A \rangle$

✦ En Java **no existe directamente**, pero se simula con *wildcards*:

$List\langle ? \text{ super } String \rangle$

Esto significa:

- Puede ser  $List\langle String \rangle$
- o  $List\langle Object \rangle$

Se usa cuando **solo se insertan valores**, no se leen.

---

### ✓ Resumen conceptual (perfecto para examen)

- $List\langle String \rangle$  **NO** es subtipo de  $List\langle Object \rangle$  → los genéricos son **invariantes**
- $String[]$  **SÍ** es subtipo de  $Object[]$  → los arrays son **covariantes**
- La covarianza de los arrays provoca errores en **tiempo de ejecución** (`ArrayStoreException`)

- Los genéricos evitan esos errores **rechazando el código en compilación**
- Definiciones:
  - **Covariante**: preserva el subtipado
  - **Contravariante**: invierte el subtipado
  - **Invariante**: no lo propaga

#### 👉 Frase redonda final:

*Java acepta la covarianza insegura de los arrays por compatibilidad, pero diseña sus genéricos como invariantes para garantizar seguridad de tipos en compilación.*

13. Java permite recuperar covarianza y contravarianza en tipos genéricos de forma controlada mediante **wildcards**. ¿Qué es un wildcard (?)? Muestra la diferencia entre `List<? extends T>` y `List<? super T>`, indicando en qué casos se usa cada uno. Pon dos ejemplos: (i) un método que reciba una lista de números y calcule su suma, usando `? extends`; (ii) un método que reciba una lista y le añada varios números enteros, usando `? super`.

Vamos a verlo con calma, porque **los wildcards (?) son la clave para recuperar covarianza y contravarianza en los genéricos de Java** de forma **segura y controlada**. Es una de las partes más importantes del tema.

### ✅ ¿Qué es un **wildcard (?)**?

Un **wildcard (?)** representa **un tipo desconocido** dentro de un tipo genérico.

No significa "cualquier tipo sin más", sino:

*"Algún tipo que cumple ciertas restricciones".*

Ejemplo básico:

```
List<?> lista;
```

Aquí:

- `lista` es una lista de **algún tipo desconocido**
- Sabemos que existen elementos, pero **no sabemos de qué tipo son**

Esto permite **flexibilizar** el subtipado sin romper la seguridad de tipos.

### ✅ Dos wildcards clave en Java

Java define dos formas principales de wildcard **acotado**:

1. `? extends T` → *covarianza*

## 2. ? super T → contravarianza

---

# ◆ List<? extends T> — Covarianza controlada

---

¿Qué significa?

```
List<? extends T>
```

Es una lista de:

- T
- o cualquier subtipo de T

Por ejemplo:

```
List<? extends Number>
```

puede ser:

- List<Integer>
- List<Double>
- List<BigDecimal>
- etc.

---

✅ ¿Cuándo se usa?

👉 Cuando **SOLO** necesitas **LEER** datos

No te importa el tipo concreto, solo que sea un subtipo de T.

---

❌ Restricción importante

No puedes añadir elementos (salvo null):

```
List<? extends Number> l;  
l.add(3); // ❌ NO compila
```

Porque Java **no puede garantizar** que 3 sea compatible con el tipo real de la lista.

---

💡 Regla clásica

## Producer Extends

(si produce valores, usa *extends*)

### ✓ Ejemplo (i): sumar una lista de números usando *? extends*

```
public static double suma(List<? extends Number> numeros) {  
    double total = 0.0;  
    for (Number n : numeros) {  
        total += n.doubleValue();  
    }  
    return total;  
}
```

Uso:

```
List<Integer> li = List.of(1, 2, 3);  
List<Double> ld = List.of(1.5, 2.5);  
  
System.out.println(suma(li));  
System.out.println(suma(ld));
```

- ✓ El mismo método funciona con **cualquier subtipo de *Number***
- ✓ Sin casting
- ✓ 100% seguro en compilación

## ◆ *List<? super T>* — Contravarianza controlada

¿Qué significa?

```
List<? super T>
```

Es una lista de:

- *T*
- o **cualquier supertipo de *T***

Por ejemplo:

```
List<? super Integer>
```

puede ser:

- `List<Integer>`
- `List<Number>`
- `List<Object>`

✅ ¿Cuándo se usa?

👉 Cuando SOLO necesitas ESCRIBIR (añadir) datos

Sabes que puedes añadir elementos de tipo `T`, porque todos sus supertipos los aceptan.

❌ Limitación

Al leer:

- Solo puedes tratar los elementos como `Object`

```
Object o = lista.get(0); // ✅  
Integer i = lista.get(0); // ❌
```

💡 Regla clásica

### Consumer Super

(si consume valores, usa `super`)

✅ Ejemplo (ii): añadir enteros a una lista usando `? super`

```
public static void añadirEnteros(List<? super Integer> lista) {  
    lista.add(1);  
    lista.add(2);  
    lista.add(3);  
}
```

Uso:

```
List<Integer> li = new ArrayList<>();  
List<Number> ln = new ArrayList<>();  
List<Object> lo = new ArrayList<>();  
  
añadirEnteros(li);  
añadirEnteros(ln);  
añadirEnteros(lo);
```

- ✓ Funciona con todas
- ✓ El compilador garantiza que es seguro
- ✓ No hay errores en tiempo de ejecución

## 🗨 Comparación clara entre ambos wildcards

| Wildcard     | Permite leer como | Permite añadir | Uso típico      |
|--------------|-------------------|----------------|-----------------|
| ? extends T  | T                 | ✗ No           | Leer / procesar |
| ? super T    | Object            | ✓ Sí           | Insertar datos  |
| ? (sin cota) | Object            | ✗ No           | Uso muy general |

## ✓ ¿Por qué existen los wildcards?

Porque los genéricos en Java son **invariantes**, y sin wildcards:

```
List<Integer> ≠ List<Number>
```

Los wildcards recuperan:

- **Covarianza** (*extends*)
- **Contravarianza** (*super*)

pero **sin romper la seguridad de tipos** (a diferencia de los arrays).

## ✓ Resumen perfecto para examen

- Un **wildcard** (?) representa un tipo desconocido.
- ? extends T:
  - permite **covarianza**
  - se usa cuando la estructura **produce valores**
- ? super T:
  - permite **contravarianza**
  - se usa cuando la estructura **consume valores**
- Regla **PECS**:
  - *Producer* → *extends*
  - *Consumer* → *super*
- Los wildcards permiten flexibilidad sin errores en tiempo de ejecución.

### 👉 Frase redonda final:

*Los wildcards permiten expresar variancia en genéricos de Java de forma controlada, trasladando los errores al compilador en lugar de al tiempo de ejecución.*